COL 1101

LECTURE 13

AUGUST 20, 2026

FUNCTIONAL STYLE PROGRAMMING

LOGISTICS

- Mark your conflict for Sep 2
  (Programming guy)

LAST CLASS: FUNCTIONAL PROGRAMMING

An alternate programming paradigm, where data is

immutable, behaviour is implemented ~~through~~ pure functions
and their composition

LAST CLASS: FUNCTIONAL PROGRAMMING

An alternate programming paradigm, where data is

immutable, behaviour is implemented ~~through~~ pure functions
(and their composition)

Pure Function :

1). output depends only on its input

2) it produces no side effects - No change that outlives the function .

LAST CLASS: FUNCTIONAL PROGRAMMING

An alternate programming paradigm, where data is immutable, behaviour is implemented through pure functions

Pure Function:

1). output depends only on its input

2) it produces no side effects - No change that outlives the function.

Example side effect: mutating arguments, global or shared state, does I/O (e.g. print), senses the outside world.

LAST CLASS: FUNCTIONAL PROGRAMMING

An alternate programming paradigm, where data is immutable, behaviour is implemented through pure functions

Pure Function:

1). output depends only on its input

2) it produces no side effects - No change that outlives the function.

Example:

```
append4(l):
    return l + [4]
```

```
greet(name)
    return "hi " + name
```

Implementing Data Structures The FP Way

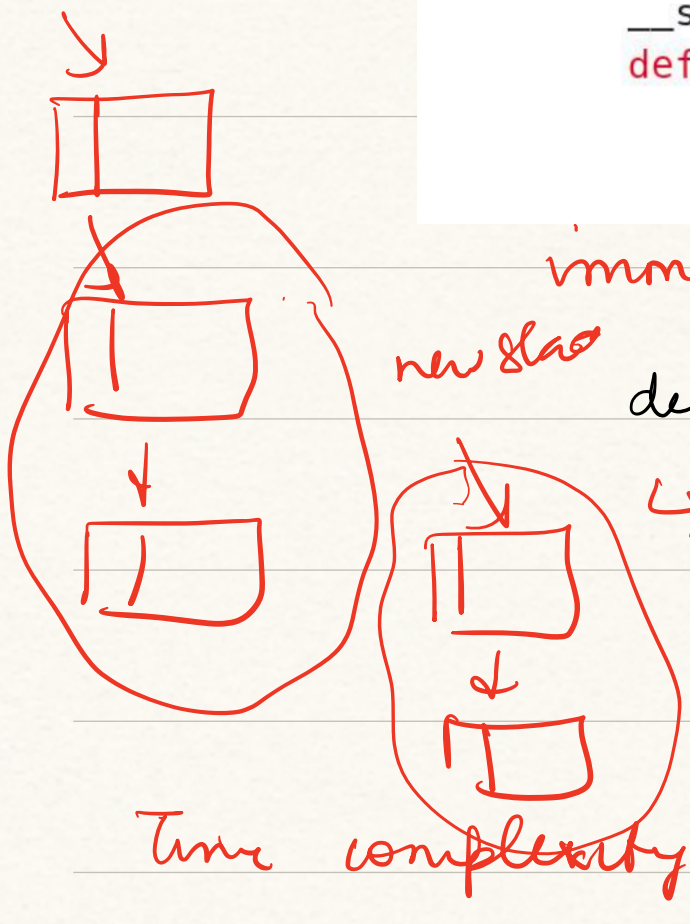
```
class _Node:
    __slots__ = ("value", "next")
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

```
def push(self, x):          # prepend
    self._head = self._Node(x, self._head)
    self._size += 1
```

How do we modify push function?

Implementing Data Structures The FP Way

```
class Node:
    __slots__ = ("value", "next")
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```



immutable

new stack

```
def push(stack, x):
    ↳ new_stack = deep_copy(stack)
    new_stack = Node(x, new_stack)
    return new_stack
```

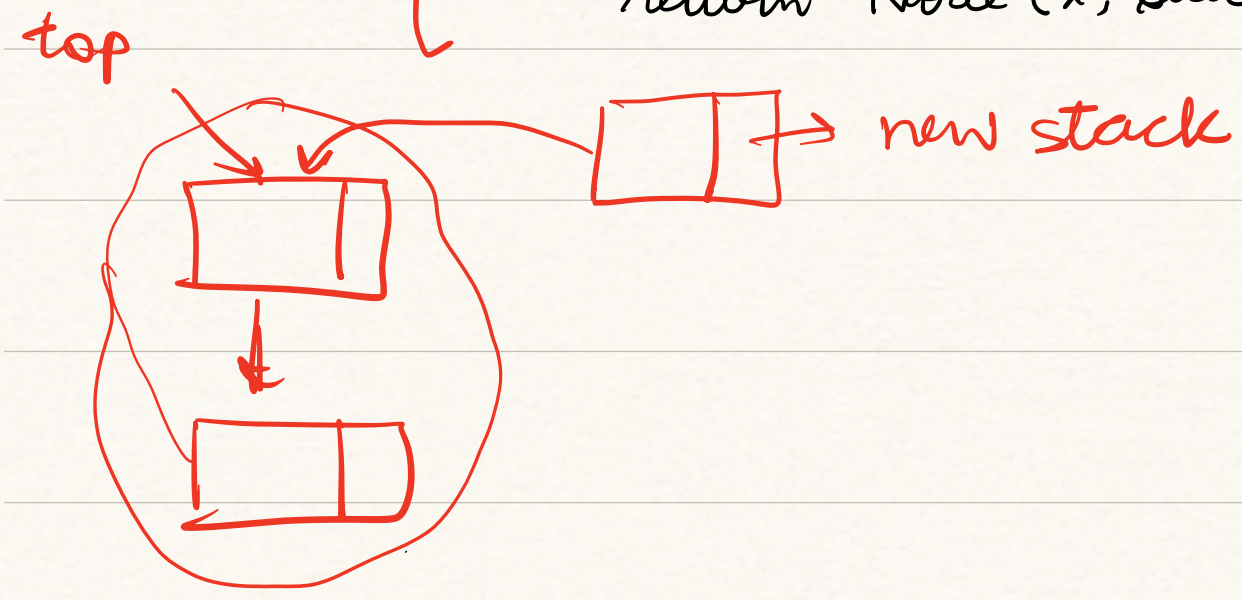
Time complexity: $\Theta(n)$

Can we do better?

Implementing Data Structures The FP Way

```
class Node:
    __slots__ = ("value", "next")
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

```
def push(stack, x):
    return Node(x, stack)
```



Implementing Data Structures The FP Way

```
class _Node:
    __slots__ = ("value", "next")
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

```
def pop(stack):
    if stack is None:
        return None
    return stack.value, stack.next
```

OOP vs FP

OOP

```
s = LinkedStack()
s.push('a'); s.push('b')
backup = s.deep_copy()
s.push('c')
backup.size()
```

↓

incorrect

Functional Programming

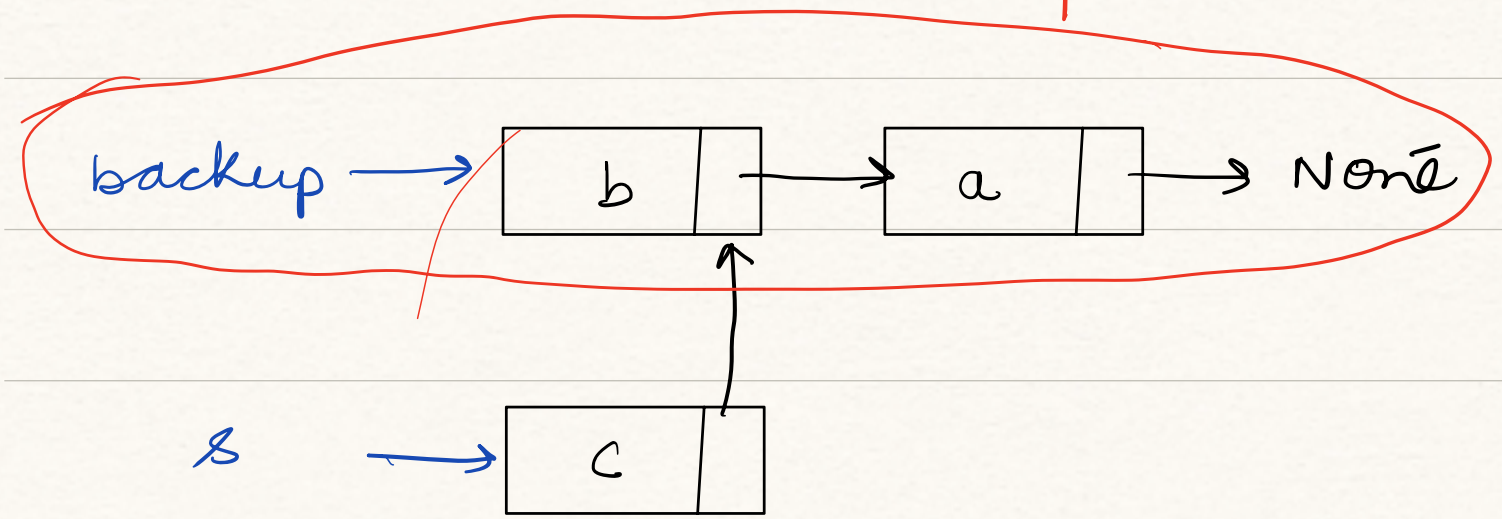
```
s = None
s = push(s, 'a')
s = push(s, 'b')
backup = s
s = push(s, 'c')
size(backup)
```

↓

2

WHAT JUST HAPPENED?

① structural sharing




```
s = push(backup, 'c')
```

[creates a new node,
tail is shared]

② persistence

: any operations do not effect old copies of data

Implementing Data Structures The FP Way

```
class Node:
    __slots__ = ('value', 'next')
    def __init__(self, value, nxt):
        self.value = value
        self.next  = nxt
```

Time complexity ?

```
def push(stack, x):
    return Node(x, stack)
```

→ $\Theta(1)$

```
def pop(stack):
    return stack.value, stack.next
```

→ $\Theta(1)$

Implementing Data Structures The FP Way

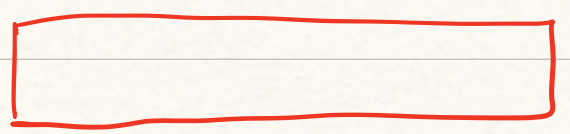
```
class Node:
    __slots__ = ('value', 'next')
    def __init__(self, value, nxt):
        self.value = value
        self.next  = nxt
```

```
def push(stack, x):
    return Node(x, stack)

def pop(stack):
    return stack.value, stack.next
```

How about using an array?

Can you do an immutable add in $\Theta(1)$



copy the array and then add

Implementing Data Structures The FP Way

```
class Node:
    __slots__ = ('value', 'next')
    def __init__(self, value, nxt):
        self.value = value
        self.next  = nxt
```

```
def push(stack, x):
    return Node(x, stack)

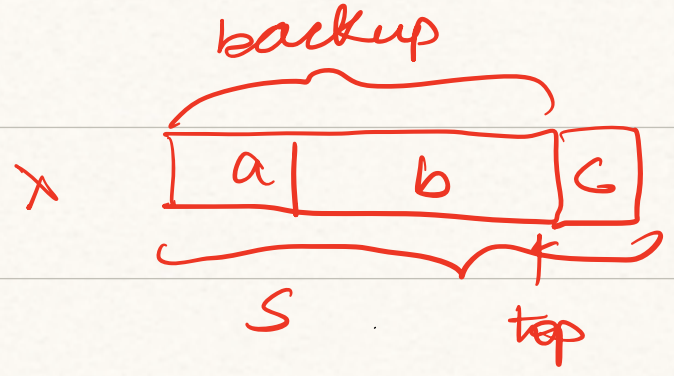
def pop(stack):
    return stack.value, stack.next
```

How about using an array?

- An immutable add on an array needs to copy the entire slab

push - O(n)

pop. - O(n)



push (backup, x)

What About Other Operations? (E.g. size)

OOP way - store the size

```
class LinkedStack:
    class _Node:
        __slots__ = ("value", "next")
        def __init__(self, value, next=None):
            self.value = value
            self.next = next

    def __init__(self):
        self._head = None
        self._size = 0

    def size(self):
        return self._size
```

def size(stack):

traverse the stack & count the elements

How to implement the above in the FP paradigm?

SIZE FUNCTION IMPLEMENTED IN THE FP PARADIGM

recursion

```
def size(stack):
    n = 0
    node = stack
    while node != None:
        n += 1
    return n
```

pure function

```
def size(stack):
    if stack is None:
        return 0
    return 1 + size(stack.next)
```

base case

Pure function but imperative style

→ Do this, then do this

imperative paradigm

transform values from values

functional style

OTHER FUNCTIONS : RECURSION

compute the sum of all values of a list

```
def total(lst):
    if lst is None:
        return 0
    return lst.value +
           total(lst.next)
```

returns True if x is present in lst, False otherwise

```
def member(lst, x):
    if lst is None:
        return False
    if x == lst.value:
        return True
    return member(lst.next,
                  x)
```

FACT : FP USES RECURSION AS THE CONTROL STRUCTURE

- No mutable state : recursion carries state in arguments and return value

- Easier to reason about the code

FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable
- algorithms are standalone functions with no hidden state - pure functions
- build programs by composing functions, not mutating objects.

FACT: FUNCTION IS JUST AN OBJECT

```
def square(x):
    return x * x
```

```
f = square
```

print(f)
print(f(5))

```
funcs = [square, len, str]
```

Functions can take functions as input and can also output functions or both

Higher-order function

HIGHER ORDER FUNCTIONS : FUNCTION AS INPUT

higher order funct — linked list

```
def list_map(f, lst):                    # apply f to every element
    if is_empty(lst):
        return empty
    return append(list_map(f, lst.next), f(lst.value))
```

push

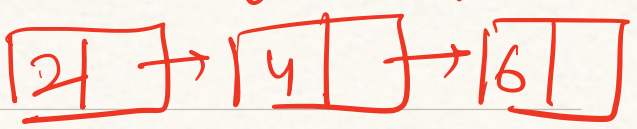
```
nums = append(append(append(None, 3), 2), 1)

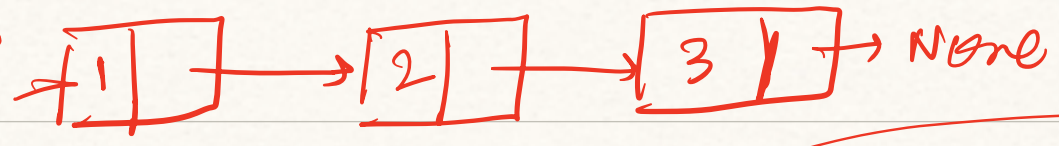
doubled = list_map(double, nums)

squared = list_map(square, nums)
```

```
def double(x):
    return 2*x

def square(x):
    return x*x
```





append(list_map(double, 2 → 3), 2)

HIGHER ORDER FUNCTIONS : FUNCTION AS INPUT

Write a function that traverses a list and filters elements satisfying a predicate (<T> → boolean)

```
def filter_list (lst, predicate):
    if lst is None:
        return None
    if predicate (lst.value):
        return append (filter2list (lst.next, predicate), lst.value)
    else
        return filter_list (lst.next, predicate)
```

HIGHER ORDER FUNCTIONS : FUNCTIONS COMING OUT

```
def adder(n):
    def add_n(x):
        return x + n
    return add_n

add5  = adder(5)
add10 = adder(10)
print(add5(100))
print(add10(100))
```

→ return x + n

→ 105

→ 110

HIGHER ORDER FUNCTIONS : FUNCTIONS IN AND OUT

```
def compose(f, g):
    def h(x):
        return f(g(x))
    return h

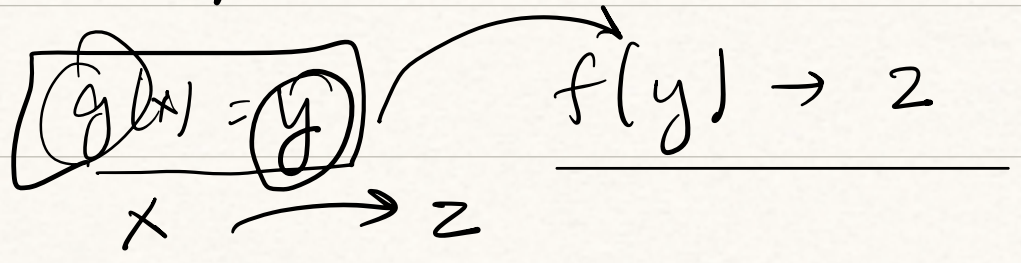
inc  = lambda x: x + 1
add6 = compose(add5, inc)
print(add6(0))
```

→ 6

CLAIM: f(g(x)) is a pure function if f & g are pure

↳ , output is the same for a given input

↳ no side effects



LAMBDA FUNCTION IN PYTHON

syntax for writing functions in a concise manner

lambda arguments : expressions

E.g. lambda x : x * x <-> [def square(x)
                            return x * x]

↓

lambda calculus

FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable
- algorithms are standalone functions with no hidden state - pure functions
- build programs by composing functions, not mutating objects.

BRINGING EVERYTHING TOGETHER

$f(g(x)) = g(f(x))$

compose (f, g)
= f(g(x))

→ keep_even = lambda s: list_filter (lambda x: x % 2 == 0, s)

→ square_all = lambda s: list_map (lambda x: x*x, s)

→ even_squares = compose (keep_even , square_all)
f g

→ sum_even_squares = compose (total , even_squares))

↳ return sum of all elems

```
nums = append (append (append (append (None, 4), 3, 2, 1)
print (sum_even_squares (nums))
```

[1, 2, 3, 4]
↓
[1, 4, 9, 16]
[4, 16] → 20